



SZKOŁA GŁÓWNA HANDLOWA W WARSZAWIE
Kolegium Analiz Ekonomicznych

Podyplomowe Studia Data Science w biznesie

Prognostowanie popytu na rowery miejskie w Londynie z wykorzystaniem
modeli regresyjnych

Imię i nazwisko autora:
Małgorzata Ciepka
Nr albumu: 138053

Praca końcowa napisana pod kierunkiem
mgr Ilona Pietras

Warszawa 2025

Spis treści

1.	Wstęp	5
1.1	Bezpieczeństwo rowerzystów	5
1.2	Popularność i rozwój systemów rowerów miejskich	6
2.	Cel i zakres pracy	7
3.	Podstawy teoretyczne	7
3.1	Regresja i predykcja	7
3.2	Drzewa decyzyjne, lasy losowe i boosting	7
4.	Materiał i metodyka pracy	8
4.1	Zbiór danych	8
4.2	Proces budowania modeli w języku Python	11
4.2.1	Wczytanie danych	11
4.2.2	Przygotowanie danych do dalszej analizy	12
4.2.3	Inżynieria cech	12
4.2.4	Eksploacyjna analiza danych	14
4.3	Budowa modelu regresyjnego - Las Losowy (Random Forest Regressor)	23
4.3.1	Optymalizacja modelu danych – lagging	25
4.3.2	Budowa modelu lasu losowego na danych po laggingu	27
4.3.3	Optymalizacja hiperparametrów	28
4.3.4	Budowa modelu regresyjnego - ekstremalny gradient boosting (XGBoost Regressor)	29
4.3.5	Wytrenowanie modelu XGBoost	29
4.3.6	Ewaluacja modelu XGBoost	30
4.3.7	Optymalizacja hiperparametrów	31
5.	Porównanie modeli	32
5.1	Porównanie ważności cech	33

5.2	Porównanie dopasowania predykcji obu modeli do danych rzeczywistych	34
	ID najpopularniejszych stacji rowerów miejskich w Londynie:	35
5.2.1	Stacja id 191: Hyde Park Corner, Hyde Park.....	35
5.3	Stacja id 154: Waterloo Station 3, Waterloo Station	36
5.4	Stacja id 303: Albert Gate, Hyde Park	36
5.4.1	Stacja id 307: Black Lion Gate, Kensington Garden	37
5.4.2	Stacja id 374: Waterloo Station 6, Waterloo Station	37
6.	Podsumowanie i wnioski.....	38
6.1	Wnioski:	38
7.	Źródła danych.....	39
8.	Spis literatury	39
9.	Spis ilustracji	41
10.	Spis tabel	43

1. Wstęp

Od kilku lat, w europejskich miastach, lokalne władze stawiają na zrównoważone formy transportu. Nie jest to tylko związane z emisją gazów cieplarnianych, ale również z takimi wyzwaniami jak zanieczyszczenie powietrza, hałas czy bezpieczeństwo mieszkańców. Jednym z elementów, które stają się coraz powszechniejsze, są systemy rowerów miejskich tzw. bike-sharing.

Londyn jest jednym z przodowników rozwoju takich systemów. Jeszcze w latach sześćdziesiątych XX wieku, w centrum Londynu powszechne były tzw. autostrady miejskie – szerokie wielopasmowe ulice, gdzie priorytetowym użytkownikiem byli kierowcy samochodów. Ulice takie jak Lower Thames Street oraz Upper Thames Street świadczyły o dominacji samochodów w mieście i były skrajnie nieprzyjazne innym użytkownikom dróg. W przypadku rowerzystów, tylko nieliczni byli na tyle odważni, aby jeździć miejskimi autostradami – jedynymi śmiałkami byli kurierzy rowerowi. Dopiero w 2014 roku pojawił się projekt, aby na tych ulicach pojawiły się drogi dla rowerów, tzw. rowerowe drogi szybkiego ruchu z oddzieloną krawężnikami przestrzenią dla rowerzystów, z bezpiecznymi skrzyżowaniami i wydzielonymi fazami świateł. Projekt budził wiele kontrowersji, gdzie najczęściej powtarzaniem argumentem było to, że te drogi będą całkowicie puste poza godzinami szczytu. Rowerostrady zostały otwarte w 2015 roku i od razu poskutkowały znaczącym przyrostem rowerzystów na Lower i Upper Thames Street, w szczególności osób starszych czy rodziców z dziećmi. Jazda na rowerze zaczęła być postrzegana jako nie tylko sport, ale również tanim i wygodnym, alternatywnym środkiem transportu.

1.1 Bezpieczeństwo rowerzystów

Głównym argumentem przeciwko jeździe na rowerze są samochody. Wiele badań wskazuje na to, że społeczeństwo się po prostu boi. W 2010 roku, w miastach Leicester, Leeds, Worcester i Lancaster, zostały przeprowadzone wśród rodzin badania o wyborach związanych z poruszaniem się w mieście. Częstą odpowiedzią był to, że badani „[...]nie wyobrażają sobie podróżowania rowerem w środowisku miejskim tak silnie zorientowanym na samochody”¹. W 2015 roku, zespół z Uniwersytetu Westminster, przeprowadził badanie według którego ponad 80% badanych deklarowało co najmniej jeden niebezpieczny

¹ Walker P., „Jak rowery mogą uratować świat”, wyd. Wysoki Zamek, Kraków, 2018

incydent dziennie, który ich spotkał w trakcie jazdy na rowerze. Trzy czwarte przypadków było powiązanych z samochodami. Poprzez wprowadzanie takich rozwiązań jak zmniejszanie prędkości na małych ulicach do 30 km/h lub mniejszej, poprawia nie tylko bezpieczeństwo rowerzystów, ale również pieszych.

1.2 Popularność i rozwój systemów rowerów miejskich

Pierwowzorem miejskich wypożyczalni rowerów był ruch „Biały Rower” – zapoczątkowana przez anarchistów inicjatywa w Amsterdamie w 1965 roku. Na terenie miasta zostały porozstawiane nieprzypięte białe rowery, z których mógł skorzystać każdy. Inicjatywa nie była wielkim sukcesem, jednak gdy holenderska policja postanowiła skonfiskować te rowery, co zapoczątkowało dość charakterystyczny protest, w trakcie którego demonstranci rzucali w funkcjonariuszy górnymi połówkami dzwonków rowerowych. Ruch „Biały Rower”, mimo niepowodzenia, zaczął obrastać w mit, a postulaty z manifestu grupy, min. o tym jak niebezpiecznym i nieprzystosowanym do miasta środkiem transportu są samochody, zaczęły nabierać popularności, a w Europie zaczęły pojawiać się kolejne małe inicjatywy rowerów miejskich.

Nowoczesne systemy rowerów miejskich zaczęły pojawiać się w połowie lat dwutysięcznych – pierwszy był system Velo’v w Lyonie w 2005 roku.. 2007 rok to systemy w Paryżu i Krakowie, w 2008 roku dołączył Montreal, Waszyngton i Hangzhou. 2010 rok to początki systemów rowerów miejskich w Meksyku, Buenos Aires oraz Londynie. Przed inauguracją systemu rowerów miejskich, wśród Londyńczyków panował duży sceptycyzm. Duże obawy budziło, jak sobie będą radzić z rowerami turyści i początkujący rowerzyści, ale te obawy się nie spełniły, a system bike-sharingu chwali sobie nawet pierwszy Brytyjczyk, który zwyciężył Tour de France - Bradley Wiggins.

Czynniki, które sprawiają, że popularność miejskich wypożyczalni rowerów wciąż rośnie to m. in.:

- Dostępność
- Niskie koszty
- Ekologia
- Rozwój infrastruktury miejskiej w tym bezpiecznych dróg dla rowerów
- Pozytywny wpływ aktywności ruchowej na zdrowie

2. Cel i zakres pracy

Celem pracy jest opracowanie dwóch modeli regresyjnych – lasu losowego oraz ekstremalnego gradient boostingu w celu prognozy dziennego popytu na rowery na stacjach systemu rowerów miejskich w Londynie, na podstawie danych historycznych, pogodowych oraz dotyczących bezpieczeństwa rowerzystów w mieście.

3. Podstawy teoretyczne

W niniejszym rozdziale omówione zostaną podstawowe pojęcia dotyczące modeli regresyjnych, które zostały wykorzystane w niniejszej pracy do prognozowania popytu na rowery miejskie w Londynie.

3.1 Regresja i predykcja

W erze Big Data regresja zaczęła być wykorzystywana do budowania modeli predykcyjnych, a nie tylko objaśniania zebranych danych. Najprostszą formą regresji jest regresja liniowa, która zakłada liniową zależność między zmiennymi. Równanie regresji modeluje związek pomiędzy zmienną celu/objaśnianą Y , a zmienną objaśniającą X . Model regresji dostarcza wartości aproksymowanych (predykcji) i ich rezyduów (błędów predykcji)

3.2 Drzewa decyzyjne, lasy losowe i boosting

Drzewa decyzyjne (ang. decision trees) są metodą opartą na uczeniu zagregowanym (ang. ensemble learning). Podstawową zasadą tego typu machine learningu jest użycie wielu modeli predykcyjnych. Kombinacja uczenia zagregowanego z metodami drzew decyzyjnych to jedna z najwydajniejszych technik modelowania predykcyjnego. Nazywane również drzewami regresji i klasyfikacji (ang. Classification and Regression Trees), są popularną metodą klasyfikacji stworzoną w 1984 przez Leo Breimana i zespół oraz stanowią podstawy najszerzej wykorzystywanych i skutecznych narzędzi do modelowania predykcyjnego dla regresji klasyfikacji.

Las losowy (ang. Random Forest) jest metodą opartą na baggingu dla drzew decyzyjnych rozszerzoną nie tylko o próbkowanie rekordów – algorytm próbkuje również zmienne objaśniające podczas podziałów. Praktyczny wynik lasu losowego jest miarą istotności zmiennych, która szereguje predyktory w zależności od ich udziału w dokładności modelu.

Boosting, czyli wzmacnianie to ogólna technika dopasowania sekwencji modeli poprzez dodawanie większych wag rekordom z większymi resztami przy każdym skutecznym dopasowaniu. Najpopularniejszym oprogramowaniem dla metod boostingu to XGBoost, który jest wydajną obliczeniowo implementacją stochastycznego wzmacniania gradientowego z wieloma opcjami.

4. Materiał i metodyka pracy

4.1 Zbiór danych

Dane wykorzystane w pracy pochodzą z kilku źródeł. Pierwsze, najważniejsze źródło danych dotyczy systemu rowerów miejskich w Londynie, pochodzi z publicznych repozytoriów danych dostępnych w Google BigQuery, do analizy zostały wykorzystane dane z lat 2016-2018. Zbiór ten został poszerzony o dane o bezpieczeństwie drogowym rowerzystów w Londynie pochodzące z portalu Kaggle oraz informacje o warunkach pogodowych pochodzące z portalu Visual Crossing, dokładne źródła zostały opisane w bibliografii oraz załączonym do pracy notatniku Python.

Przed przystąpieniem do analizy danych i budowy modeli predykcyjnych, powyżej wymienione dane zostały połączone w jeden zbiór danych - do każdej stacji rowerów miejskich zostały przypisane kolizje/wypadki z udziałem rowerzystów w promieniu 500 m od danej stacji na podstawie danych geograficznych. Dane zostały połączone za pomocą języka SQL w środowisku Google BigQuery, kod znajduje się poniżej:

```
WITH acc AS
(
SELECT
    a.accident_date,
    s.id AS station_id,
    s.name AS station_name,
    SUM(a.is_fatal) AS fatal_acc,
    SUM(a.is_serious) AS serious_acc,
    SUM(a.is_slight) AS slight_acc,
    COUNT(a.accident_ID) AS num_acc,
FROM `malgocie.bikes.accidents` AS a
INNER JOIN `bigquery-public-data.london_bicycles.cycle_stations` AS s ON
ST_DWITHIN(ST_GEOGPOINT(s.longitude, s.latitude),
ST_GEOGPOINT(a.accident_longitude, a.accident_latitude), 500)
```



```

WHERE EXTRACT(YEAR FROM a.accident_date) IN (2016, 2017, 2018)
GROUP BY 1,2,3
)

SELECT
  DATE(r.start_date) AS date,
  s.id AS station_id,
  s.name AS station_name,
  h.is_holiday,
  COUNT(r.rental_id) AS trip_count,
  CASE
    WHEN SUM(acc.fatal_acc) IS NOT NULL THEN SUM(acc.fatal_acc)
    ELSE 0
  END
  AS fatal_acc,
  CASE
    WHEN SUM(acc.serious_acc) IS NOT NULL THEN SUM(acc.serious_acc)
    ELSE 0
  END
  AS serious_acc,
  CASE
    WHEN SUM(acc.slight_acc) IS NOT NULL THEN SUM(acc.slight_acc)
    ELSE 0
  END
  AS slight_acc,
  CASE
    WHEN SUM(num_acc) IS NOT NULL THEN SUM(num_acc)
    ELSE 0
  END
  AS num_acc,
  w.temp1,
  w.temp2,
  w.hum,
  w.wind_speed,
  w.weather_type,
  w.weather_code
FROM `bigquery-public-data.london_bicycles.cycle_hire` AS r

```

```

LEFT JOIN acc ON DATE(r.start_date)=acc.accident_date AND
r.start_station_id=acc.station_id
LEFT JOIN `malgocie.bikes.uk-bank-holidays` AS h ON
DATE(r.start_date)=h.date
LEFT JOIN `malgocie.bikes.weather` AS w ON DATE(r.start_date)=w.date
LEFT JOIN `bigquery-public-data.london_bicycles.cycle_stations` AS s ON
r.start_station_name=s.name
WHERE EXTRACT(year FROM DATE(r.start_date)) IN (2016, 2017, 2018)
GROUP BY ALL
ORDER BY date, station_id

```

Powstały zbiór danych składa się z 785518 wierszy oraz 15 kolumn. Informacje o kolumnach:

- **date** – data wypożyczenia roweru miejskiego
- **station_id** – unikalne id stacji wypożyczeni
- **station_name** – nazwa stacji wypożyczeni
- **is_holiday** – informacja, czy w danym dniu występuje dzień ustawowo wolny od pracy tzw. bank holiday
- **trip_count** – liczba wypożyczeń rowerów
- **fatal_acc** – liczba wypadków śmiertelnych z udziałem rowerzysty/rowerzystów
- **serious_acc** – liczba kolizji z udziałem rowerzysty/rowerzystów, w których była co najmniej jedna osoba ranna
- **slight_acc** – liczba kolizji z udziałem rowerzysty/rowerzystów, w których żadna z osób uczestniczących nie została ranna
- **num_acc** – suma wszystkich wypadków i kolizji z udziałem rowerzysty/rowerzystów
- **temp1** – średnia dobową temperatura w stopniach Celsjusza
- **temp2** – średnia dobową temperatura odczuwalna w stopniach Celsjusza
- **hum** – średnia wilgotność powietrza w procentach
- **wind_speed** – średnia dobową prędkość wiatru w kilometrach na godzinę
- **weather_type** – ogólna informacja o pogodzie
- **weather_code** – zakodowana ogólna informacja o pogodzie (1 – słonecznie, 2 – częściowe zachmurzenie, 3 – pełne zachmurzenie, 4 – deszcz, 5 – śnieg)

Gotowy zbiór danych został dołączony do niniejszej pracy jako plik *london_bikes.csv*.

4.2 Proces budowania modeli w języku Python

Do dalszych prac został wykorzystany język Python wraz z odpowiednimi bibliotekami. Do pracy został załączony plik Jupyter Notebook *Prognozowanie_popytu_rowery_miejskie_Londyn.ipynb*, w którym znajduje się całość kodu wraz z opisem. W dalszej części pracy zostaną omówione najważniejsze fragmenty kodu.

Wykorzystane biblioteki:

- **Pandas** – biblioteka służąca do manipulacji i analizy danych
- **Numpy** – obsługa wielowymiarowych tabel oraz macierzy
- **Seaborn** – biblioteka służąca do wizualizacji danych
- **Matplotlib** – podstawowa biblioteka służąca do wizualizacji danych
- **Datetime** – obsługa różnych formatów dat
- **Sci-kit Learn** – biblioteka wykorzystywana do budowy modeli machine learningowych w Python oraz ich ewaluacji
- **Xgboost** – biblioteka służąca do budowy modeli extra gradient boosting

4.2.1 Wczytanie danych

W pierwszej kolejności zostały wczytane biblioteki konieczne do analizy danych – pandas, numpy, seaborn, matplotlib.pyplot oraz datetime.

```
# Załadowanie potrzebnych bibliotek
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from datetime import date
```

Następnie do DataFrame'u został wczytany wcześniej utworzony zbiór danych *london_bikes.csv* oraz wyświetlone pierwsze pięć wierszy zbioru danych.

```
# Wczytanie zbioru danych
df = pd.read_csv('london_bikes.csv')

# Sprawdzenie pierwszych pięciu wierszy zbioru danych
df.head()
```

	date	station_id	station_name	is_holiday	trip_count	fatal_acc	serious_acc	slight_acc	num_acc	temp1	temp2	hum	wind_speed	weather_type	weather_code
0	2016-01-01	NaN	NaN	1	716	0	0	0	0	5.9	3.6	84.7	24.7	rain	4
1	2016-01-01	1.0	River Street , Clerkenwell	1	7	0	0	0	0	5.9	3.6	84.7	24.7	rain	4
2	2016-01-01	2.0	Phillimore Gardens, Kensington	1	10	0	0	0	0	5.9	3.6	84.7	24.7	rain	4
3	2016-01-01	4.0	St. Chad's Street, King's Cross	1	3	0	0	0	0	5.9	3.6	84.7	24.7	rain	4
4	2016-01-01	5.0	Sedding Street, Sloane Square	1	16	0	0	0	0	5.9	3.6	84.7	24.7	rain	4

Rys. 1 Widok pięciu pierwszych wierszy wgranych danych

4.2.2 Przygotowanie danych do dalszej analizy

Po sprawdzeniu podstawowych informacji o zbiorze danych – typy danych, liczba wierszy oraz kolumn, kolejnym krokiem było zarządzanie brakami danych.

```
# Sprawdzenie, czy w zbiorze danych są brakujące wartości
df.isna().sum()
```

Po wyświetleniu braków okazało się, że w kolumnach *station_id* oraz *station_name* brakuje po 1096 wartości, ze względu na ich niewielką liczbę w stosunku do wielkości zbioru, 1096 wierszy z brakami danych zostało usuniętych.

```
# Ze względu na małą liczbę brakujących wartości w stosunku do całego
zbioru danych, usuniemy je
df.dropna(inplace=True)
```

Następnie sprawdzone zostało istnienie duplikujących wartości, które nie wystąpiły w danym zbiorze.

```
# Sprawdzenie czy w zbiorze danych są duplikaty
duplicates = df.duplicated().sum()
print(f'Liczba duplikatów: {duplicates}')
```

Liczba duplikatów: 0

4.2.3 Inżynieria cech

Po usunięciu braku danych oraz sprawdzeniu duplikujących się wartości, następnym krokiem była inżynieria cechu w celu przygotowania zbioru pod eksploracyjną analizę danych oraz budowę modeli machine learningowych.

Zmiana typu danych kolumny *is_holiday* na boolean.

```
# Konwersja kolumny 'is_holiday' na typ boolean
df.is_holiday = df.is_holiday.astype('bool')
```

Utworzenie nowych kolumn z informacjami dzień, miesiąc, rok, numer dnia tygodnia na podstawie kolumny *date*.

```
# Stworzenie nowej kolumny 'is_weekend' wskazującej czy dany dzień to weekend
df['is_weekend'] = df.datet.dt.dayofweek >= 5
```

Utworzenie nowej kolumny z informacją o porze roku na podstawie nowoutworzonej kolumny *month*.

```
# Stworzenie nowej kolumny 'season' wskazującej na porę roku
def get_season(month):
    if month in [12, 1, 2]:
        return 1 #coding for winter
    elif month in [3, 4, 5]:
        return 2 #coding for spring
    elif month in [6, 7, 9]:
        return 3 #coding for summer
    else:
        return 4 #coding for autumn

df['season'] = df.datet.dt.month.apply(get_season)
```

Kody poszczególnych pór roku:

- 1 – zima
- 2 – wiosna
- 3 – lato
- 4 – jesień

Następnie ponownie zostały sprawdzone typy danych.

```
# Sprawdzenie typów zmiennych w zbiorze
print('\nData types:\n', df.dtypes)
```

Data types:

date	object
station_id	float64
station_name	object
is_holiday	bool
trip_count	int64
fatal_acc	int64
serious_acc	int64
slight_acc	int64

```

num_acc          int64
temp1            float64
temp2            float64
hum              float64
wind_speed       float64
weather_type     object
weather_code     int64
datet            datetime64[ns]
year             int32
month            int32
day              int32
weekday          int32
is_weekend       bool
season           int64
dtype: object

```

4.2.4 Eksploracyjna analiza danych

Kolumny zostały podzielone według typów zmiennych na zmienne numeryczne oraz zmienne katégoryczne (skokowe).

```

# Określenie typów zmiennych dla każdej kolumny
categorical_features = ['weekday', 'is_holiday', 'weather_code',
                        'is_weekend', 'season']
numerical_features = ['trip_count', 'fatal_acc', 'serious_acc',
                      'slight_acc', 'num_acc', 'temp1', 'temp2', 'hum', 'wind_speed']

```

Następnie zostały wyświetlone statystyki opisowe dla zmiennych numerycznych oraz ich histogramy. Na tym etapie nie wykryto żadnych anomalii.

```

# Wyświetlenie statystyk opisowych dla zmiennych numerycznych
df[numerical_features].describe()

```

	trip_count	fatal_acc	serious_acc	slight_acc	num_acc	temp1	temp2	hum	wind_speed
count	784422.000000	784422.000000	784422.000000	784422.000000	784422.000000	784422.000000	784422.000000	784422.000000	784422.000000
mean	36.835787	0.005897	0.299042	1.622704	1.927644	12.243218	11.385072	75.045485	20.703807
std	33.843973	0.619292	5.047158	11.244247	12.408619	5.645112	6.605506	10.182179	7.450950
min	1.000000	0.000000	0.000000	0.000000	0.000000	-3.200000	-8.500000	39.600000	4.700000
25%	17.000000	0.000000	0.000000	0.000000	0.000000	7.700000	5.800000	67.800000	15.400000
50%	29.000000	0.000000	0.000000	0.000000	0.000000	11.900000	11.700000	75.500000	19.900000
75%	46.000000	0.000000	0.000000	0.000000	0.000000	16.800000	16.800000	82.700000	24.900000
max	895.000000	161.000000	821.000000	744.000000	821.000000	26.800000	26.600000	99.400000	59.000000

Rys. 2 Podstawowe statystyki danych numerycznych

```

# histogramy dla zmiennych numerycznych

```

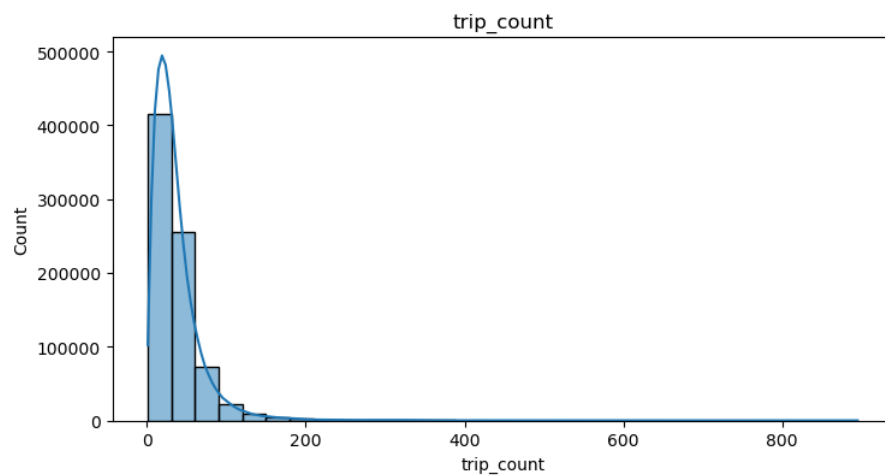
```

n_cols = 2
n_rows = int(np.ceil(len(numerical_features)/n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 20), sharey=False)
col_idx = 0

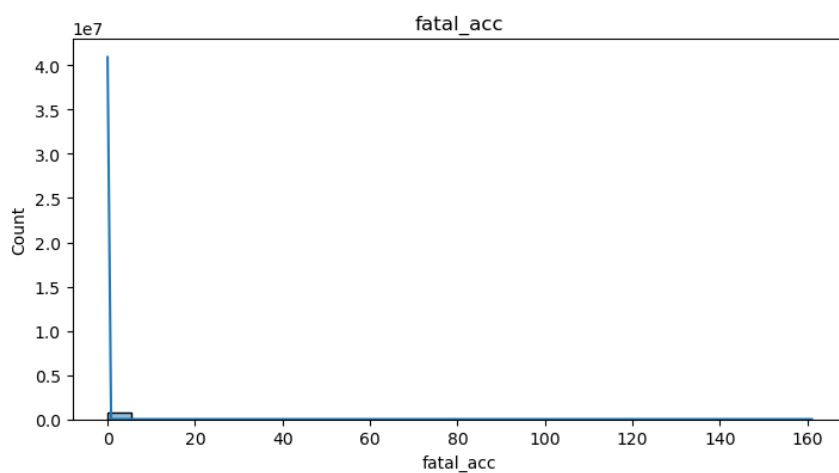
for row in range(n_rows):
    for col in range(n_cols):
        if col_idx < len(numerical_features):
            sns.histplot(df[numerical_features[col_idx]], ax = axes[row,
col], bins=30, kde=True)
            axes[row, col].set_title(numerical_features[col_idx])
            col_idx += 1
        else:
            break

if n_cols * n_rows > len(numerical_features):
    for ax in axes.flatten()[len(numerical_features):]:
        ax.remove()
plt.tight_layout()
plt.show()

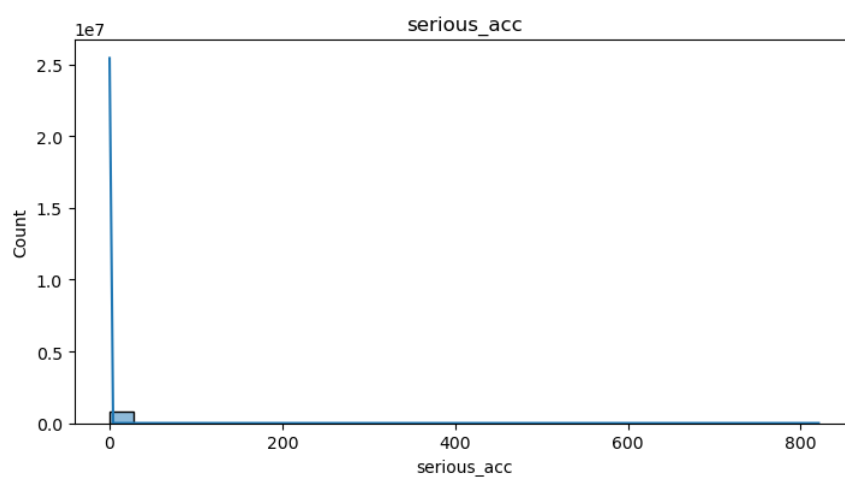
```



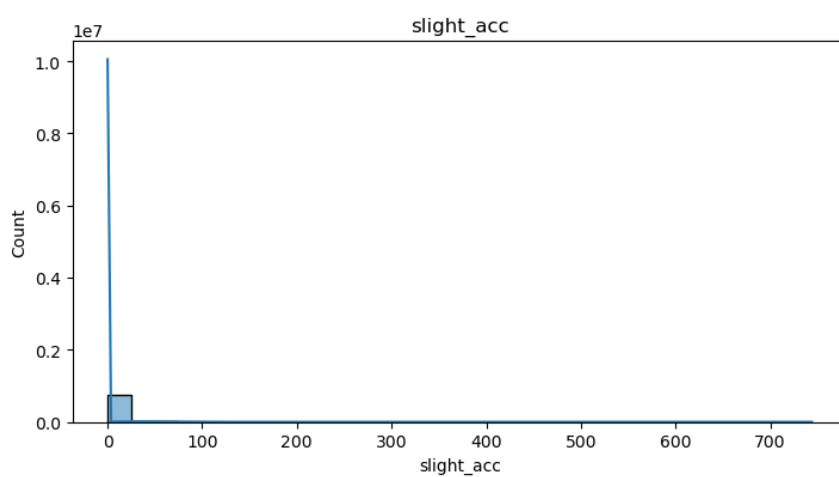
Rys. 3 Liczebność występowania zmiennej 'trip count'



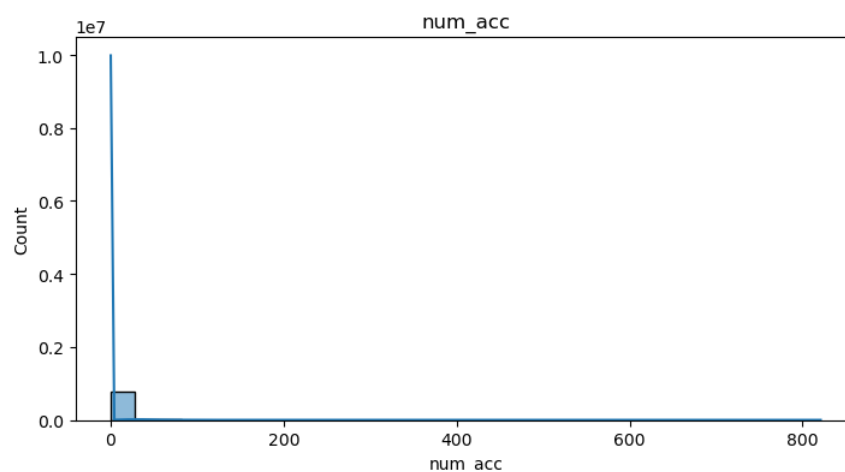
Rys. 4 Liczebność występowania zmiennej 'fatal_acc'



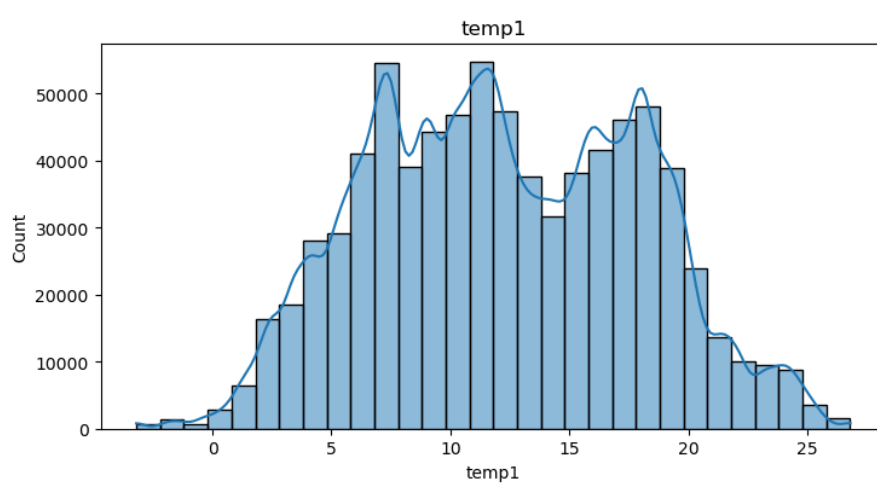
Rys. 5 Liczebność występowania zmiennej 'serious_acc'



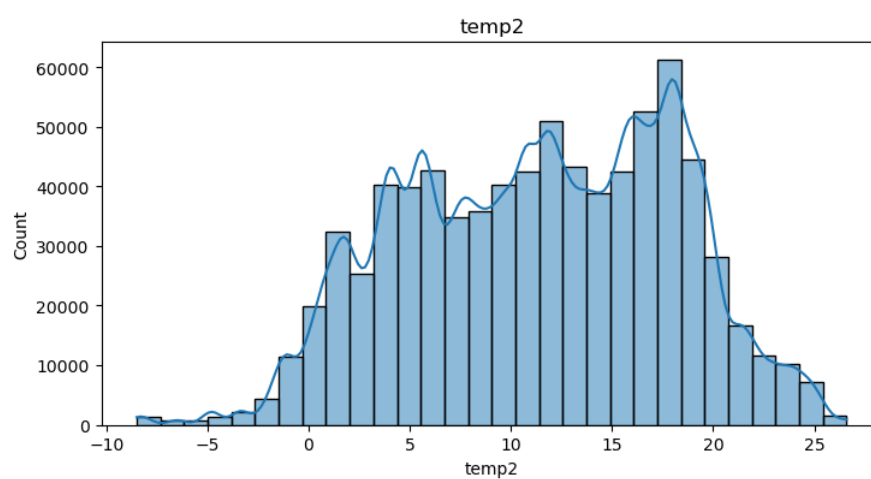
Rys. 6 Liczebność występowania zmiennej 'slight_acc'



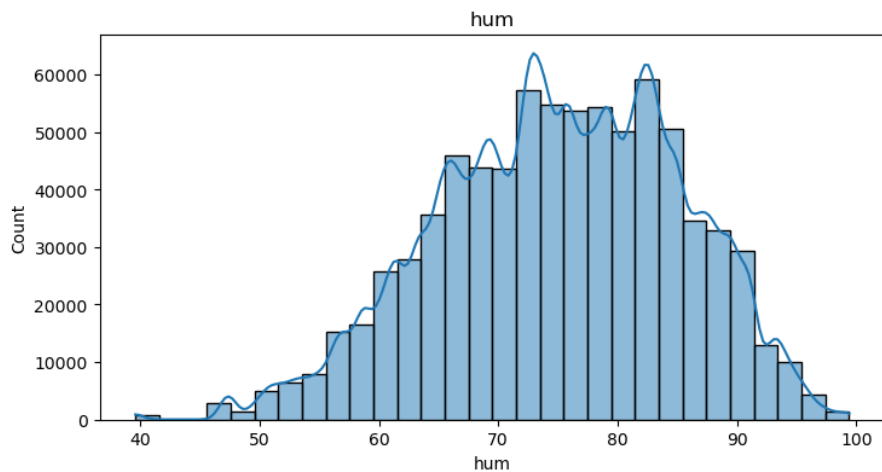
Rys. 7 Liczebność występowania zmiennej 'num_acc'



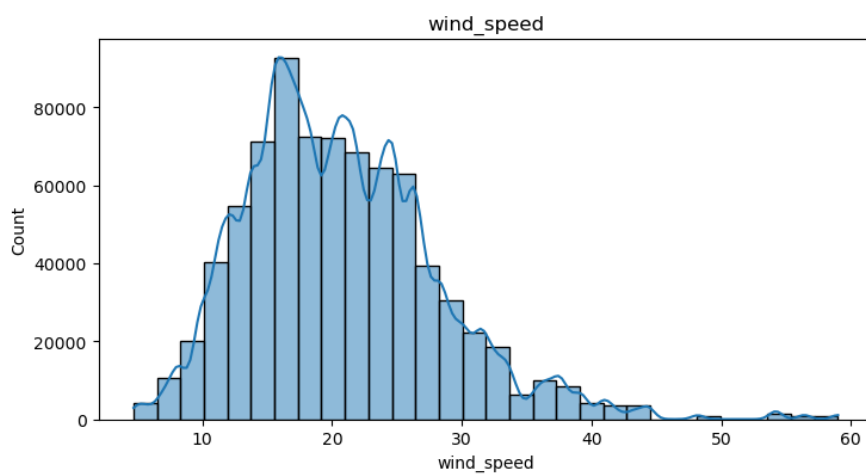
Rys. 8 Liczebność występowania zmiennej 'temp1'



Rys. 9 Liczebność występowania zmiennej 'temp2'



Rys. 10 Liczebność występowania zmiennej 'hum'



Rys. 11 Liczebność występowania zmiennej 'wind_speed'

W następnej kolejności zostały utworzone wykresy dla zmiennych skokowych, na których zostały pokazane licznosci poszczególnych kategorii.

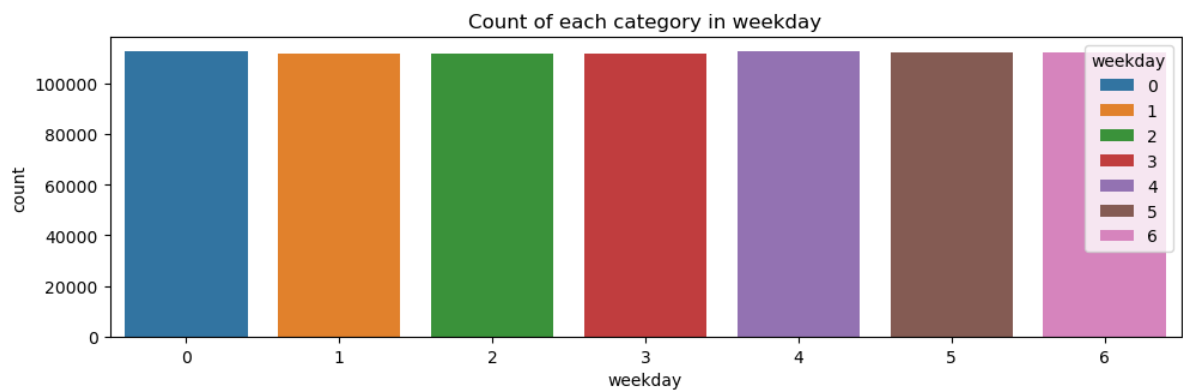
```
# Wykresy dla zmiennych kategoriycznych
n_cols = 2
n_rows=int(np.ceil(len(categorical_features)/n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 10), sharey=False)
col_idx = 0
for row in range(n_rows):
    for col in range(n_cols):
        sns.countplot(ax=axes[row, col], x=categorical_features[col_idx],
            data=df, hue=categorical_features[col_idx], palette='tab10')
        axes[row, col].set_title(f'Count of each category in
{categorical_features[col_idx]}')
        col_idx += 1
    if col_idx >= len(categorical_features):
```

```

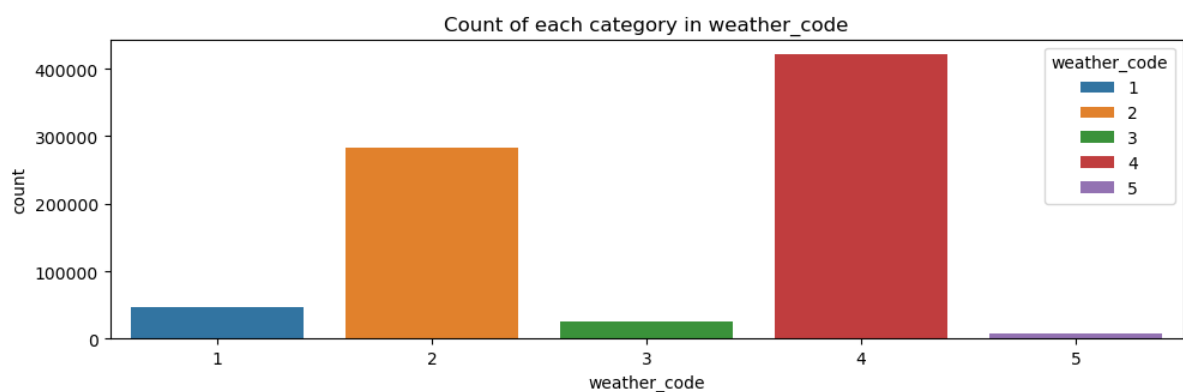
break

if n_cols*n_rows > len(categorical_features):
    for ax in axes.flatten()[len(categorical_features):]:
        ax.remove()
plt.tight_layout()
plt.show()

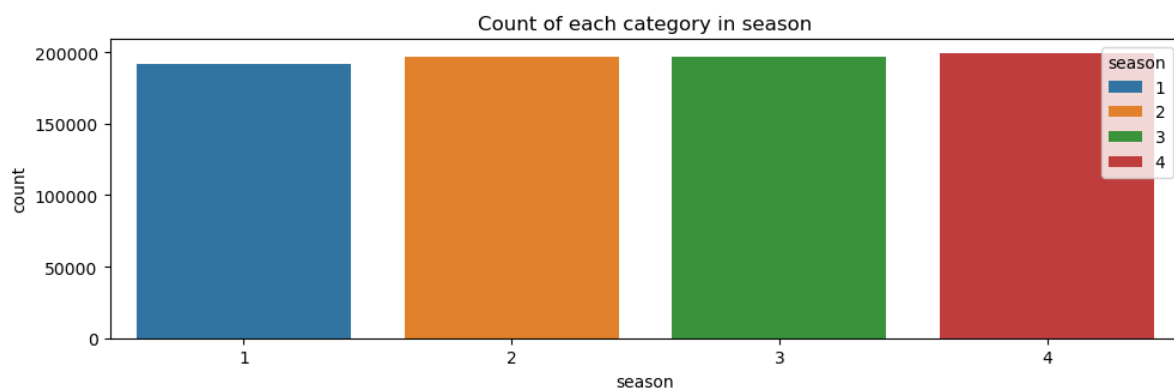
```



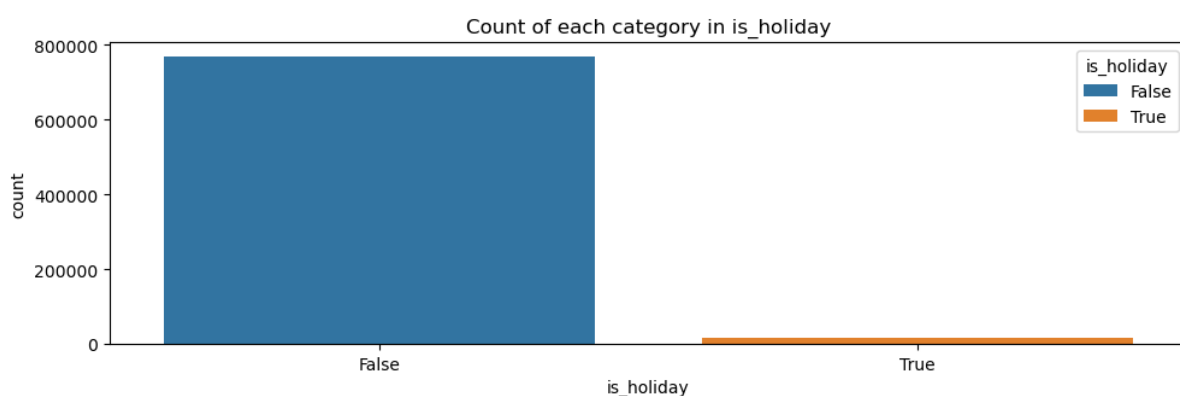
Rys. 12 Liczebność występowania poszczególnych wartości zmiennej skokowej 'weekday'



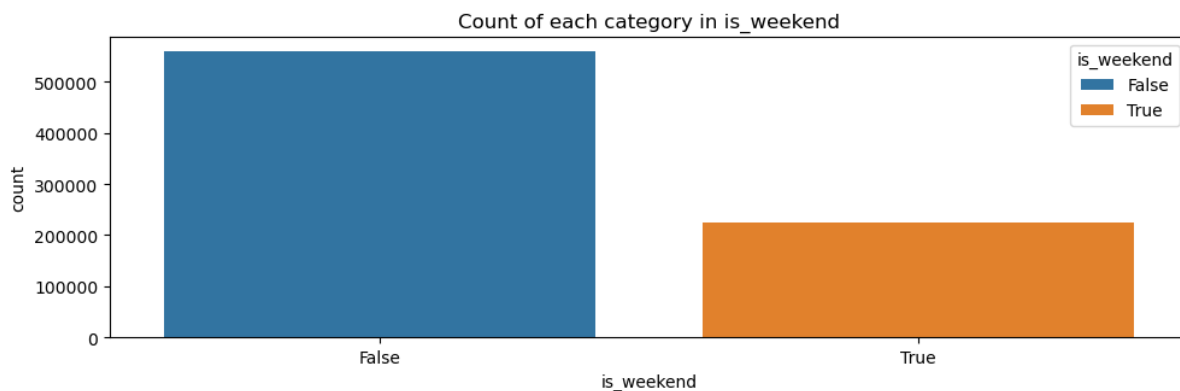
Rys. 13 Liczebność występowania poszczególnych wartości zmiennej skokowej 'weather_code'



Rys. 14 Liczebność występowania poszczególnych wartości zmiennej skokowej 'season'



Rys. 15 Liczebność występowania poszczególnych wartości zmiennej skokowej 'is_holiday'

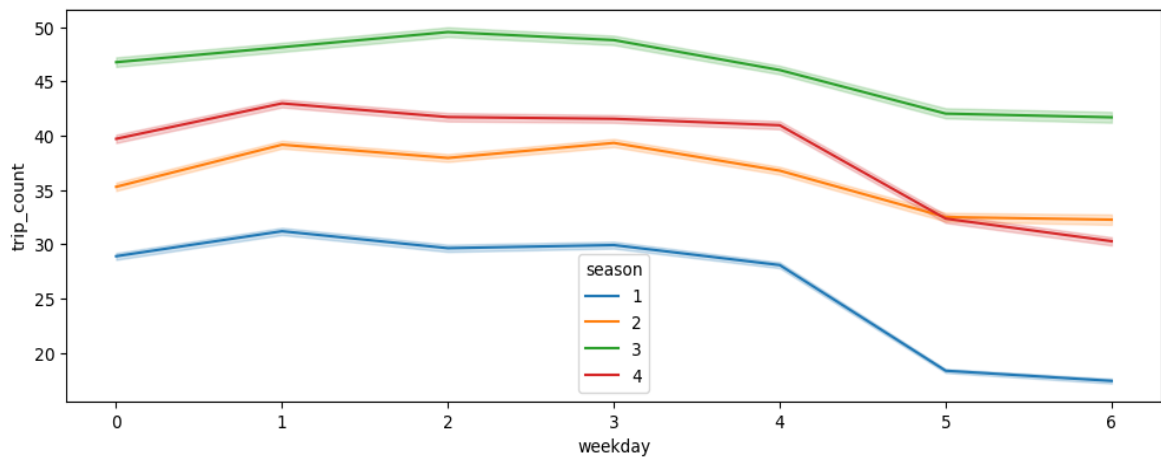


Rys. 16 Liczebność występowania poszczególnych wartości zmiennej skokowej 'is_weekend'

Następnie w sposób graficzny została przeprowadzona analiza liczby wypożyczeń w czasie – wykresy liniowe.

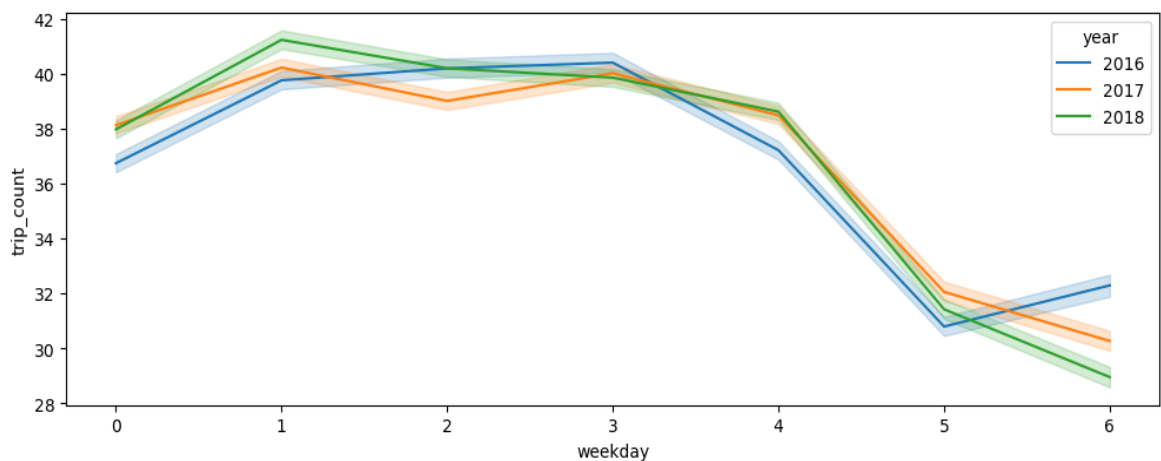
```
# Wykres liniowy liczby wypożyczeń w zależności od dnia tygodnia i pory
roku
plt.figure(figsize=(12,4))
```

```
sns.lineplot(x='weekday', y='trip_count', data=df, hue='season',
palette='tab10')
```



Rys. 17 Wykres liniowy liczby wypożyczeń w zależności od dnia tygodnia i pory roku

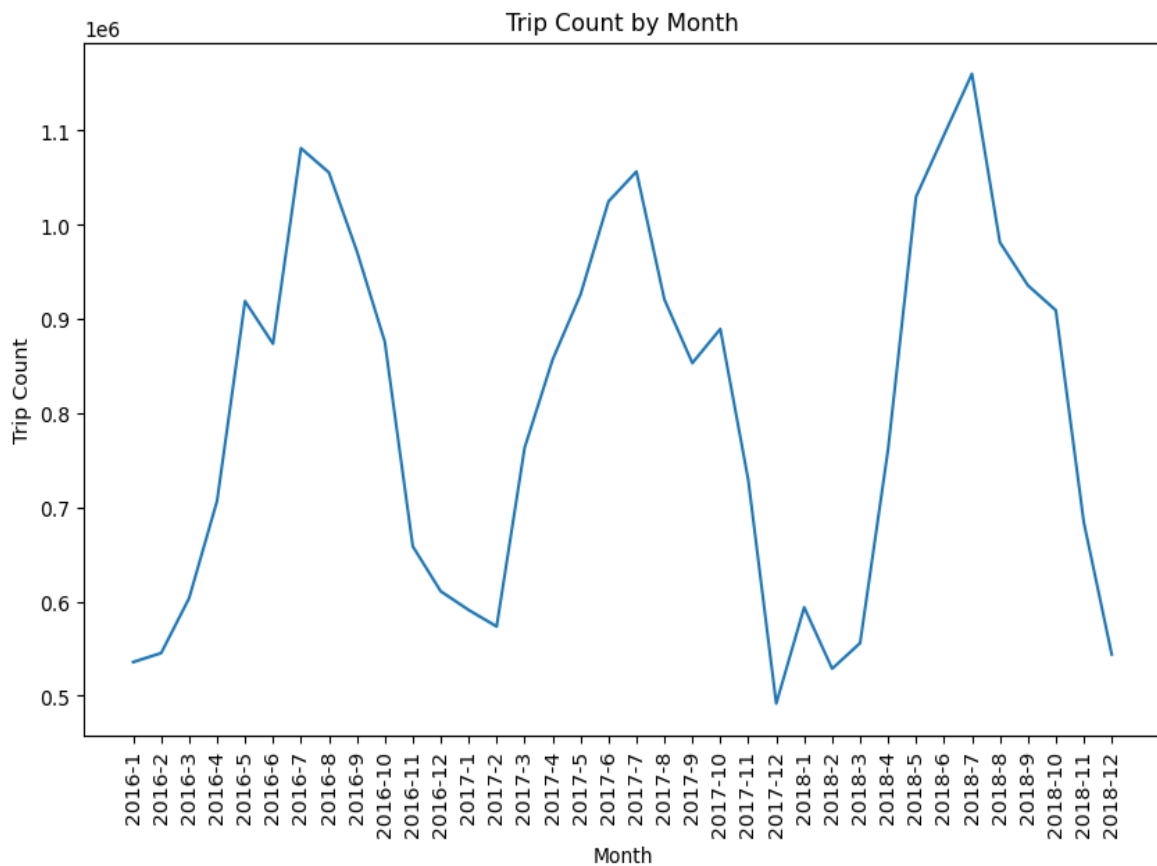
```
# Wykres liniowy liczby wypożyczeń w zależności od dnia tygodnia i roku
plt.figure(figsize=(12,4))
sns.lineplot(x='weekday', y='trip_count', data=df, hue='year',
palette='tab10')
```



Rys. 18 Wykres liniowy liczby wypożyczeń w zależności od dnia tygodnia i roku

```
# Wykres liniowy wypożyczeń przez cały zakres czasowy
df_by_m = df.groupby(['year', 'month']).agg({'trip_count':
'sum'}).reset_index()
df_by_m['year_month'] = df_by_m['year'].astype(str) + '-' +
df_by_m['month'].astype(str)
```

```
plt.figure(figsize=(10,6))
sns.lineplot(x='year_month', y='trip_count', data=df_by_m)
plt.title('Trip Count by Month')
plt.xlabel('Month')
plt.ylabel('Trip Count')
plt.xticks(rotation=90)
plt.show()
```



Rys. 19 Wykres liniowy wypożyczeń przez cały zakres czasowy

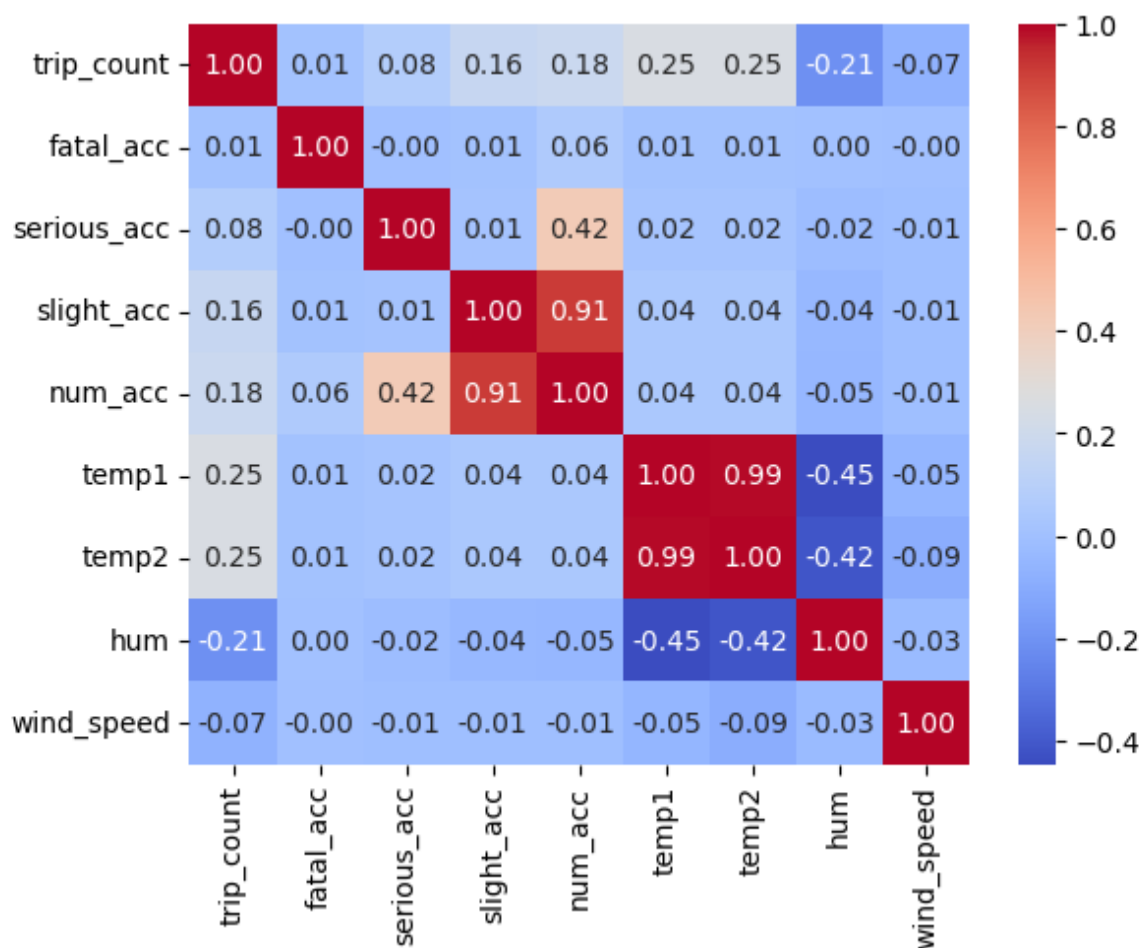
Stworzona została również tabela korelacji oraz jej graficzna reprezentacja.

```
# Tabela korelacji dla zmiennych numerycznych
correlation_matrix = df[numerical_features].corr()
correlation_matrix
```

	trip_count	fatal_acc	serious_acc	slight_acc	num_acc	temp1	temp2	hum	wind_speed
trip_count	1.000000	0.007936	0.078564	0.161815	0.178983	0.252541	0.253967	-0.208687	-0.071897
fatal_acc	0.007936	1.000000	-0.000564	0.008118	0.057035	0.006078	0.006205	0.000677	-0.004039
serious_acc	0.078564	-0.000564	1.000000	0.013866	0.419283	0.017668	0.017601	-0.017219	-0.006353
slight_acc	0.161815	0.008118	0.013866	1.000000	0.912210	0.041199	0.041200	-0.042514	-0.005983
num_acc	0.178983	0.057035	0.419283	0.912210	1.000000	0.044822	0.044802	-0.045495	-0.008207
temp1	0.252541	0.006078	0.017668	0.041199	0.044822	1.000000	0.993104	-0.445837	-0.051726
temp2	0.253967	0.006205	0.017601	0.041200	0.044802	0.993104	1.000000	-0.415808	-0.094227
hum	-0.208687	0.000677	-0.017219	-0.042514	-0.045495	-0.445837	-0.415808	1.000000	-0.029452
wind_speed	-0.071897	-0.004039	-0.006353	-0.005983	-0.008207	-0.051726	-0.094227	-0.029452	1.000000

Rys. 20 Tabela korelacji dla zmiennych numerycznych

```
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
```



Rys. 21 Heatmapa korelacji dla zmiennych numerycznych

4.3 Budowa modelu regresyjnego - Las Losowy (Random Forest Regressor)

Do budowy modelu zostały wykorzystane komponenty biblioteki Python scikit-learn.

```
# Załadowanie potrzebnych bibliotek
```

```

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn import metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score

```

Następnie zbiór danych został posortowany według daty i numeru id stacji, usunięte zostały kolumny, które nie będą wykorzystane do budowy modelu. Tematem pracy jest prognoza popytu na rowery miejskie w Londynie, a co za tym idzie, na zmienną celu została wybrana kolumna *trip_count*. Zmienne objaśniające zostały oddzielone od zmiennej celu.

```

# Posortowanie zbioru danych według daty
df.sort_values(by=['date', 'station_id'], inplace=True)

# Kolumna 'station_name' i 'weather_type' nie będą potrzebna do trenowania
modelu
df1 = df.drop(columns=['station_name', 'weather_type', 'date', 'date'])

# Oddzielenie zmiennych objaśniających od zmiennej celu
X = df1.drop('trip_count', axis=1)
y = df1['trip_count']

```

Ze względu na to, że w prognozowaniu popytu na rowery miejskie, zmienna celu jest zależna od czasu, zbiór danych został podzielony chronologicznie na zbiory treningowe oraz testowe, przy czym do trenowania modelu przeznaczono 80% zbioru danych.

```

# Chronologiczny podział danych na zbiory treningowe i testowe
split_index=int(len(X)*0.8) #80% danych do treningu
X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[:split_index], y[split_index:]

```

Następnie został wytrenowany wstępny model regresyjny lasu losowego w celu oceny czy konieczne będą dodatkowe działania na danych. Już ewaluacja modelu na zbiorze treningowym wykazała, że model jest słabej jakości – wynik R^2 wyniósł ok. 0,1045.

```

# Wytrenowanie pierwszego modelu Random Forest
rfr1 = RandomForestRegressor(n_estimators=5, max_depth=2)
rfr1.fit(X_train, y_train)

# Wstępna ewaluacja pierwszego modelu na zbiorze treningowym

```



```
rfr1.score(X_train, y_train)
```

Dla formalności została wykonana również ewaluacja na zbiorze testowym.

```
# Wytrenowanie pierwszego modelu
y_pred_rfr1 = rfr1.predict(X_test)

# Ewaluacja modelu
mse_rfr1 = mean_squared_error(y_test, y_pred_rfr1)
rmse_rfr1 = np.sqrt(mse_rfr1)
mae_rfr1 = mean_absolute_error(y_test, y_pred_rfr1)
r2_rfr1 = r2_score(y_test, y_pred_rfr1)

print('Initial Model Test Metrics:')
print(f'Mean Squared Error on test set: {mse_rfr1}')
print(f'Root Mean Squared Error on test set: {rmse_rfr1}')
print(f'Mean Absolute Error on test set: {mae_rfr1}')
print(f'R^2 Score on test set: {r2_rfr1}')
```

Wyniki ewaluacji zostały przedstawione w poniższej tabeli:

Tab. 1 Wyniki metryk ewaluacyjnych dla modelu RandomForestRegressor na zbiorze testowym

Metryka	Wartość
Średni błąd kwadratowy (MSE)	1162,2750
Pierwiastek ze średniego błędu kwadratowego (RMSE)	34,0921
Średni błąd absolutny (MAE)	21,2610
R ²	0,0949

4.3.1 Optymalizacja modelu danych – lagging

Do poprawy wyników modelu regresji została zastosowana metoda laggingu. Do zbioru danych zostały dodane kolumny z informacjami o historycznych wartościach liczby przejazdów – 1 dzień wstecz, 7 dni wstecz, 30 dni wstecz oraz 90 dni wstecz.

Przed wykonaniem laggingu, dane zostały ponownie posortowane, tym razem najpierw po numerze id stacji, a następnie dacie. Powstałe brakujące wartości zostały uzupełnione zerami i ponownie po sortowane pod budowę modelu lasu losowego – najpierw po dacie, a następnie numerze id stacji.

```

#posortowanie danych
df2 = df.sort_values(by= ['station_id', 'date'])
df2.head()

# Stworzenie nowych kolumn z lagowanymi wartościami liczby przejazdów - 1,
7, 30 i 90 dni wstecz
for i in [1, 7, 30, 90]:
    df2[f'lagged_trip_count_{i}d'] =
df2.groupby('station_id')['trip_count'].shift(i)

# Uzupełnienie brakujących wartości w nowych kolumnach
df2.fillna(0, inplace=True)

# Posortowanie zbioru danych według daty
df2.sort_values(by=['date', 'station_id'], inplace=True)

```

Powstały w ten sposób zbiór danych został przygotowany do dalszych prac poprzez ponowne usunięcie niepotrzebnych kolumn, oddzielenie zmiennej celu *trip_count* od zmiennych objaśniających oraz chronologicznym podziale danych na zbiór treningowy (80% całego zbioru danych) oraz zbiór testowy (20% całego zbioru danych) oraz został ustawiony sposób krosswalidacji modelu dla szeregów czasowych.

```

# Kolumny 'station_name', 'weather_type', 'date' oraz 'date' nie będą
potrzebna do trenowania modelu
df2 = df2.drop(columns=['station_name', 'weather_type', 'date',
'date'])

# Oddzielenie zmiennych objaśniających od zmiennej celu
X = df2.drop('trip_count', axis=1)
y = df2['trip_count']

# Chronologiczny podział danych na zbiory treningowe i testowe
split_index=int(len(X)*0.8) #80% danych do treningu
X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[:split_index], y[split_index:]

# Ustawienie TimeSeriesSplit dla walidacji krzyżowej
tscv = TimeSeriesSplit(n_splits=4)

```

4.3.2 Budowa modelu lasu losowego na danych po laggingu.

Wstępna ewaluacja nowego modelu na danych treningowych wykazała znaczną poprawę jakości modelu z wyniku R^2 0,1045 do 0,5847.

```
# Wytrenowanie drugiego modelu RandomForest
rfr2 = RandomForestRegressor(n_estimators=5, max_depth=2)
rfr2.fit(X_train, y_train)

# Wstępna ewaluacja drugiego modelu na zbiorze treningowym
rfr2.score(X_train, y_train)
```

Następnie wykonana ewaluacja modelu na danych testowych.

```
y_pred_rfr2 = rfr2.predict(X_test)

# Ewaluacja modelu
mse_rfr2 = mean_squared_error(y_test, y_pred_rfr2)
rmse_rfr2 = np.sqrt(mse_rfr2)
mae_rfr2 = mean_absolute_error(y_test, y_pred_rfr2)
r2_rfr2 = r2_score(y_test, y_pred_rfr2)

print(f'Mean Squared Error on test set: {mse_rfr2}')
print(f'Root Mean Squared Error on test set: {rmse_rfr2}')
print(f'Mean Absolute Error on test set: {mae_rfr2}')
print(f'R^2 Score on test set: {r2_rfr2}')
```

Wyniki ewaluacji zostały przedstawione w poniżej tabeli:

Tab. 2 Wyniki metryk ewaluacyjnych dla modelu RandomForestRegressor na zbiorze testowym

Metryka	Wartość
Średni błąd kwadratowy (MSE)	499,4364
Pierwiastek ze średniego błędu kwadratowego (RMSE)	22,3480
Średni błąd absolutny (MAE)	13,9154
R ²	0,6110

4.3.3 Optymalizacja hiperparametrów

W celu polepszenia parametrów modelu, została wykonana optymalizacja hiperparametrów lasu losowego za pomocą funkcji *GridSearchCV* z krosswalidacją *TimeSeriesSplit* właściwą dla szeregów czasowych.

```
# Ustawienie parametrów do grid search
param_grid = {
    'n_estimators': [5, 10, 20],
    'max_depth': [2, 5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Ustawienie GridSeatchCV
rfr2_cv = GridSearchCV(estimator=rfr2, param_grid=param_grid, cv=tscv)

# Trenowanie grid search na zbiorze treningowym
rfr2_cv.fit(X_train, y_train)

# Pozyskanie najlepszego modelu i najlepszych hiperparametrów
best_model_rfr2_cv = rfr2_cv.best_estimator_
best_params_rfr2_cv = rfr2_cv.best_params_

# Ewaluacja najlepszego modelu na zbiorze testowym
y_pred_rfr2_cv = best_model_rfr2_cv.predict(X_test)
mse_rfr2_cv = mean_squared_error(y_test, y_pred_rfr2_cv)
rmse_rfr2_cv = np.sqrt(mse_rfr2_cv)
```

```

mae_rfr2_cv = mean_absolute_error(y_test, y_pred_rfr2_cv)
r2_rfr2_cv = r2_score(y_test, y_pred_rfr2_cv)

print(f'Best parameters: {best_params_rfr2_cv}')
print(f'Mean Squared Error on test set: {mse_rfr2_cv}')
print(f'Root Mean Squared Error on test set: {rmse_rfr2_cv}')
print(f'Mean Absolute Error on test set: {mae_rfr2_cv}')
print(f'R^2 Score on test set: {r2_rfr2_cv}')

```

Najlepsze parametry modelu według *GridSearchCV*:

- **max_depth:** 10
- **min_samples_leaf:** 4
- **min_samples_split:** 2
- **n_estimators:** 20

Wyniki ewaluacji dla najlepszych parametrów modelu przedstawiono w poniższej tabeli:

Tab. 3 Wyniki metryk ewaluacyjnych dla modelu RandomForestRegressor na zbiorze testowym dla najlepszych parametrów

Metryka	Wartość
Średni błąd kwadratowy (MSE)	245,0094
Pierwiastek ze średniego błędu kwadratowego (RMSE)	15,6527
Średni błąd absolutny (MAE)	9,5632
R ²	0,8092

4.3.4 Budowa modelu regresyjnego - ekstremalny gradient boosting (XGBoost Regressor)

Do budowy modelu, oprócz wcześniej załadowanych bibliotek, została załadowana biblioteka XGBoost.

```

# Załadowanie potrzebnych bibliotek
from xgboost import XGBRegressor

```

4.3.5 Wytrenowanie modelu XGBoost

Do treningu wykorzystamy wcześniej przygotowane zbiory treningowe i testowe po przeprowadzeniu laggingu.

```
# Wytrenowanie pierwszego modelu
xgbr = XGBRegressor(random_state=42)
xgbr.fit(X_train, y_train)

# Wstępna ewaluacja drugiego modelu na zbiorze treningowym
xgbr.score(X_train, y_train)
```

Ewaluacja na zbiorze treningowym wykazała bardzo dobrą wartość wyniku $R^2 = 0,8889$, z tego względu nie optymalizujemy dalej danych, które zostały wykorzystane do wytrenowania modelu i przechodzimy do ewaluacji modelu na zbiorze testowym.

4.3.6 Ewaluacja modelu XGBoost

Poniżej przedstawiony kod służy ewaluacji modelu XGBoost na zbiorze testowym. Pod kodem znajduje się tabela, w której zostały zamieszczone wyniki.

```
y_pred_xgbr = xgbr.predict(X_test)

# Ewaluacja modelu
mse_xgbr = mean_squared_error(y_test, y_pred_xgbr)
rmse_xgbr = np.sqrt(mse_xgbr)
mae_xgbr = mean_absolute_error(y_test, y_pred_xgbr)
r2_xgbr = r2_score(y_test, y_pred_xgbr)

print(f'Mean Squared Error on test set: {mse_xgbr}')
print(f'Root Mean Squared Error on test set: {rmse_xgbr}')
print(f'Mean Absolute Error on test set: {mae_xgbr}')
print(f'R^2 Score on test set: {r2_xgbr}')
```

Tab. 4 Wyniki metryk ewaluacyjnych dla modelu XGBoostRegressor na zbiorze testowym

Metryka	Wartość
Średni błąd kwadratowy (MSE)	245,0094
Pierwiastek ze średniego błędu kwadratowego (RMSE)	15,6527
Średni błąd absolutny (MAE)	9,5632
R^2	0,8092

4.3.7 Optymalizacja hiperparametrów

Podobnie jak w przypadku lasu losowego, została wykonana optymalizacja hiperparametrów lasu losowego za pomocą funkcji *GridSearchCV* z krosswalidacją *TimeSeriesSplit*.

```
# Ustawienie parametrów do grid search
param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5, 7]
}

# Ustawienie GridSeatchCV
xgbr_cv = GridSearchCV(estimator=xgbr, param_grid=param_grid, cv=tscv)

# Trenowanie grid search na zbiorze treningowym
xgbr_cv.fit(X_train, y_train)

# Pozyskanie najlepszego modelu i najlepszych hiperparametrów
best_model_xgbr_cv = xgbr_cv.best_estimator_
best_params_xgbr_cv = xgbr_cv.best_params_

# Ewaluacja najlepszego modelu na zbiorze testowym
y_pred_xgbr_cv = best_model_xgbr_cv.predict(X_test)
mse_xgbr_cv = mean_squared_error(y_test, y_pred_xgbr_cv)
rmse_xgbr_cv = np.sqrt(mse_xgbr_cv)
mae_xgbr_cv = mean_absolute_error(y_test, y_pred_xgbr_cv)
r2_xgbr_cv = r2_score(y_test, y_pred_xgbr_cv)

print(f'Best parameters: {best_params_xgbr_cv}')
print(f'Mean Squared Error on test set: {mse_xgbr_cv}')
print(f'Root Mean Squared Error on test set: {rmse_xgbr_cv}')
print(f'Mean Absolute Error on test set: {mae_xgbr_cv}')
print(f'R^2 Score on test set: {r2_xgbr_cv}')
```

Najlepsze parametry modelu według *GridSearchCV*:

- **learning_rate:** 0.1

- **max_depth:** 7
- **n_estimators:** 50

Tab. 5 Wyniki metryk ewaluacyjnych dla modelu XGBoostRegressor na zbiorze testowym dla najlepszych parametrów

Metryka	Wartość
Średni błąd kwadratowy (MSE)	232,8627
Pierwiastek ze średniego błędu kwadratowego (RMSE)	15,2598
Średni błąd absolutny (MAE)	9,21
R ²	0,8186

5. Porównanie modeli

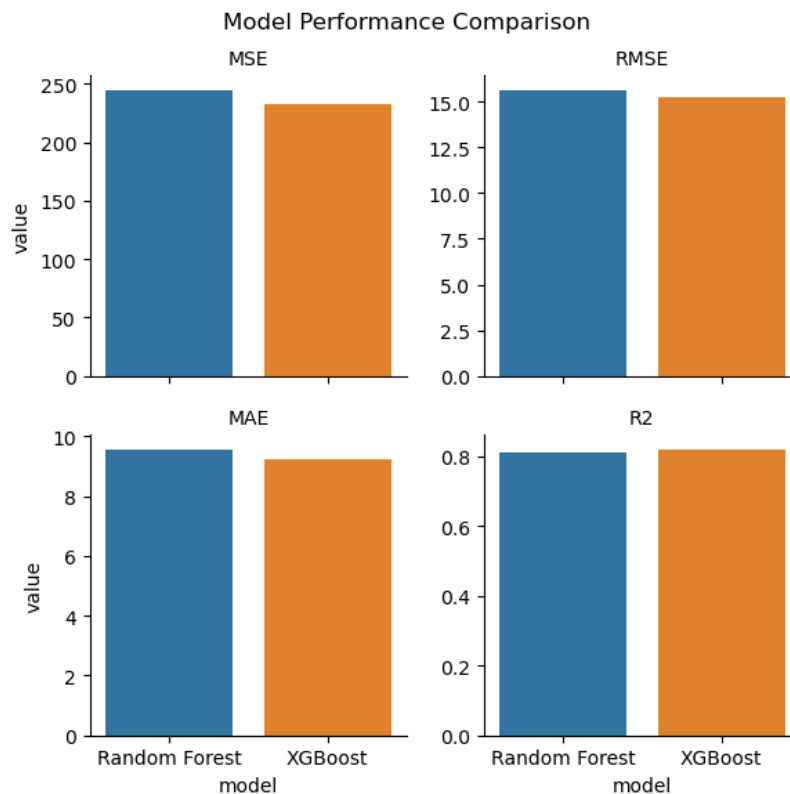
Metryki obu modeli wskazują, że zarazem Random Forest Regressor oraz XGBoost Regressor osiągnęły bardzo podobne wyniki, z minimalną przewagą na korzyść XGBoosta. Dodatkową zaletą modelu ekstremalnego gradient boostingu jest to, że podobną jakość modelu uzyskujemy w zdecydowanie krótszym czasie obliczeniowym niż las losowy.

Wyniki ewaluacji dla obu modeli zostały przedstawione w jednej tabeli.

Tab. 6 Wyniki ewaluacji końcowych modeli RandomForestRegressor i XGBoostRegressor

Metryka	RandomForestRegressor	XGBoostRegressor
Średni błąd kwadratowy (MSE)	245.0094	232.8627
Pierwiastek ze średniego błędu kwadratowego (RMSE)	15.6527	15.2598
Średni błąd absolutny (MAE)	9.5632	9.2104
R ²	0.8092	0.8186

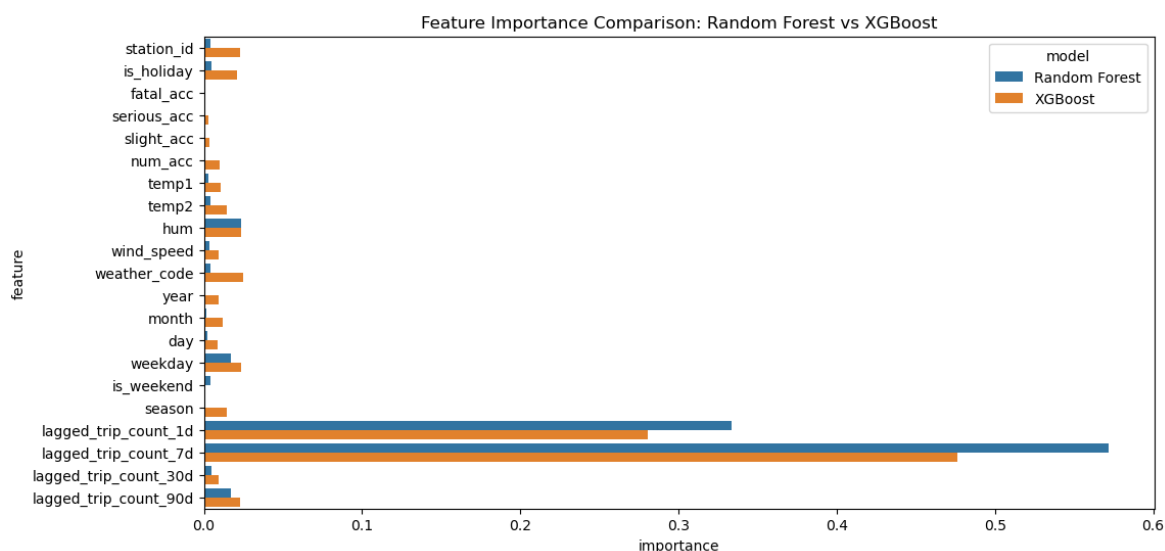
Metryki zostały również przedstawione na wykresach:



Rys. 22 Porównanie metryk obu modeli

5.1 Porównanie ważności cech

Dla obu modeli regresyjnych został stworzony wykres, na którym pokazano ważności poszczególnych zmiennych objaśniających. W przypadku Random Forest jak również XGBoosta, najbardziej istotne są cechy utworzone przez laggowanie, w szczególności opóźnienie jedno- i siedmiodniowe. Pozostałe zmienne można uznać za nieistotne.



Rys. 23 Istotność cech dla obu modeli

5.2 Porównanie dopasowania predykcji obu modeli do danych rzeczywistych

Ze względu na dużą liczbę stacji, do porównania wartości predykcji do wartości rzeczywistych wybrano 5 najbardziej popularnych stacji.

```
# Zbiór danych do sprawdzenia dopasowania wartości predykcji modeli do
wartości rzeczywistych 'trip count'
df_pred= X_test.drop(columns=['lagged_trip_count_1d',
'lagged_trip_count_7d', 'lagged_trip_count_30d', 'lagged_trip_count_90d'])
df_pred['date']=pd.to_datetime(df_pred[['year', 'month', 'day']])
df_pred['trip_count'] = y_test
df_pred['trip_count_pred_rfr'] = y_pred_rfr2_cv
df_pred['trip_count_pred_xgbr'] = y_pred_xgbr_cv

# Agregacja i filtrowanie zbioru danych do określenia pięciu
najpopularniejszych stacji
df_grouped = df_pred.groupby(['station_id']).agg({'trip_count': 'sum',
'trip_count_pred_rfr': 'sum', 'trip_count_pred_xgbr':
'sum'}).sort_values(by='trip_count', ascending=False).reset_index()
popular_stations = df_grouped['station_id'].loc[0:4]

# Wyfiltrowany zbiór danych z wartościami dla pięciu najpopularniejszych
stacji.
df_plot = df_pred[df['station_id'].isin(popular_stations)]
```

ID najpopularniejszych stacji rowerów miejskich w Londynie:

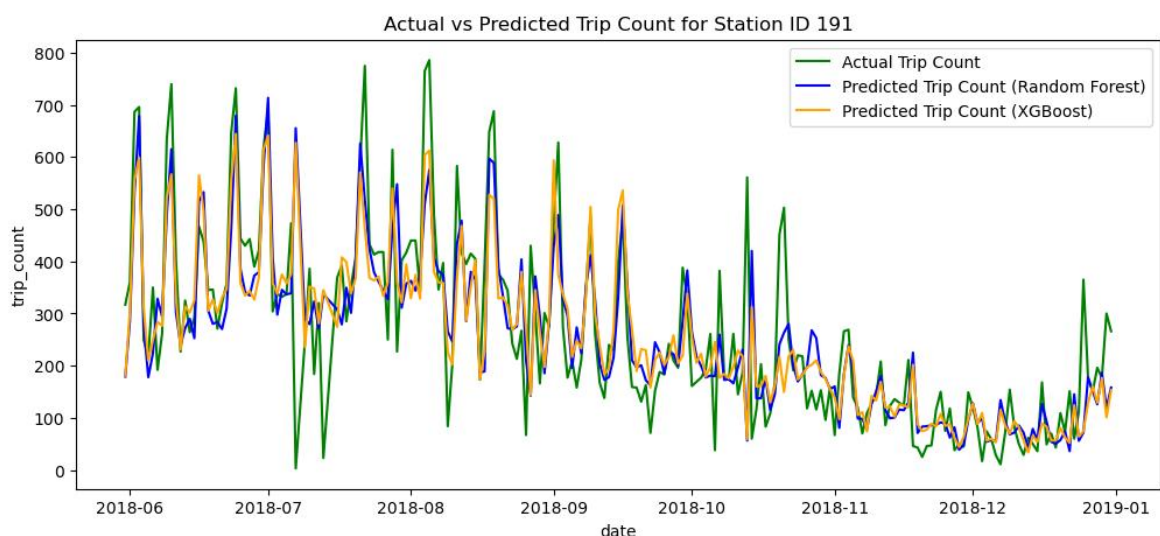
- id 191: Hyde Park Corner, Hyde Park
- id 154: Waterloo Station 3, Waterloo Station
- id 303: Albert Gate, Hyde Park
- id 307: Black Lion Gate, Kensington Garden
- id 374: Waterloo Station 6, Waterloo Station

5.2.1 Stacja id 191: Hyde Park Corner, Hyde Park

Wykres liniowy z predykcją liczby wypożyczeń rowerów miejskich przez modele Random Forest i XGBoost oraz rzeczywistą liczbą wypożyczeń.

```
station = popular_stations[0]

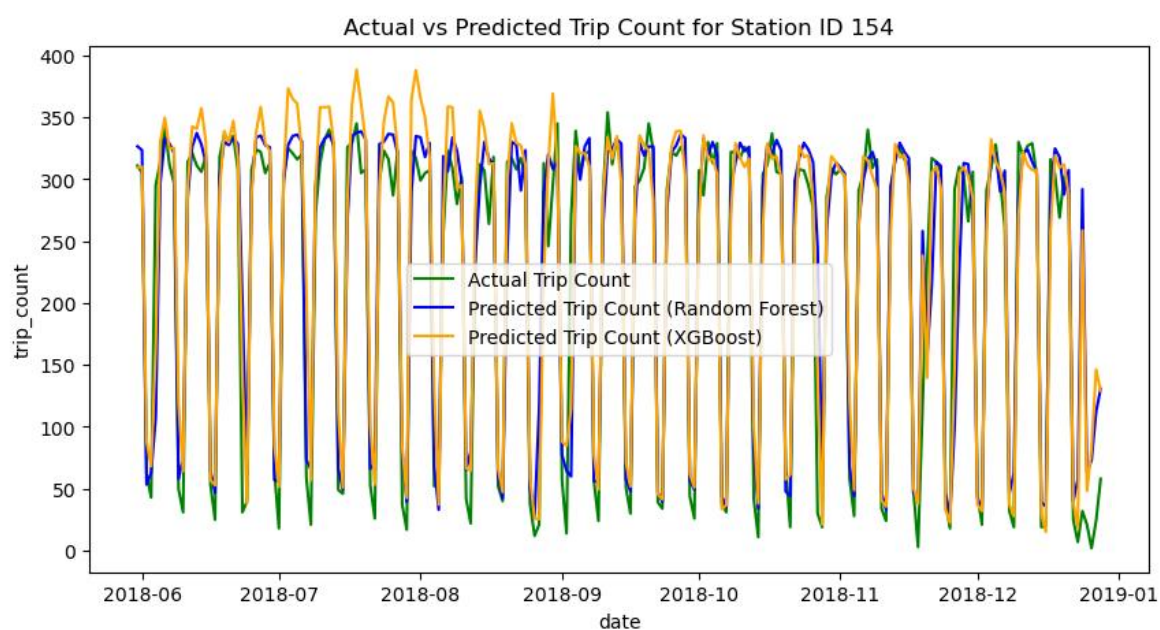
plt.figure(figsize=(10, 5))
sns.lineplot(data=df_plot[df_plot['station_id']==station], x='date',
             y='trip_count', label='Actual Trip Count', color='green')
sns.lineplot(data=df_plot[df_plot['station_id']==station], x='date',
             y='trip_count_pred_rfr', label='Predicted Trip Count (Random Forest)',
             color='blue')
sns.lineplot(data=df_plot[df_plot['station_id']==station], x='date',
             y='trip_count_pred_xgbr', label='Predicted Trip Count (XGBoost)',
             color='orange')
plt.title(f'Actual vs Predicted Trip Count for Station ID {int(station)}')
```



Rys. 24 Porównanie wyników dla stacji id 191: Hyde Park Corner, Hyde Park

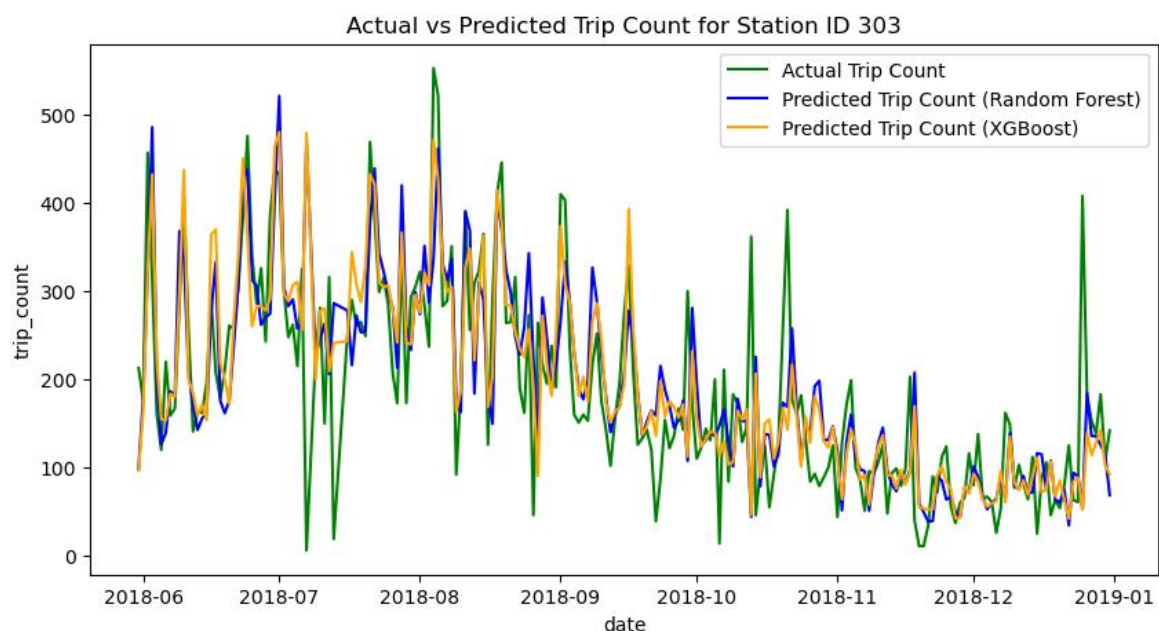
Wykresy dla kolejnych stacji zostały wykonane w ten sam sposób.

5.3 Stacja id 154: Waterloo Station 3, Waterloo Station



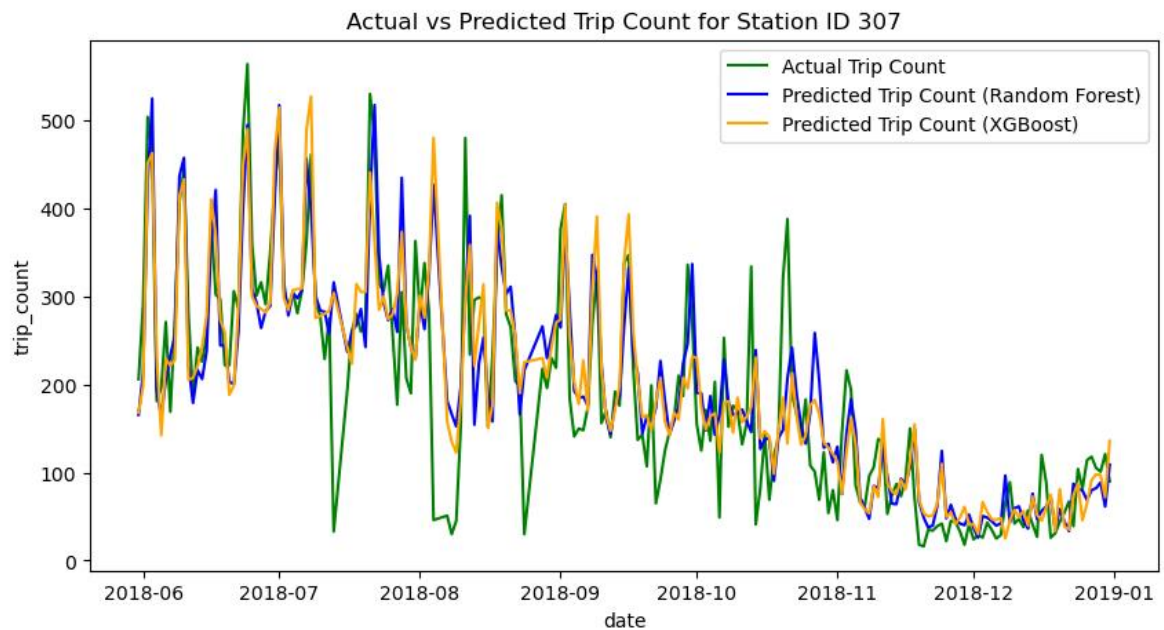
Rys. 25 Porównanie wyników dla stacji id 154: Waterloo Station 3, Waterloo Station

5.4 Stacja id 303: Albert Gate, Hyde Park



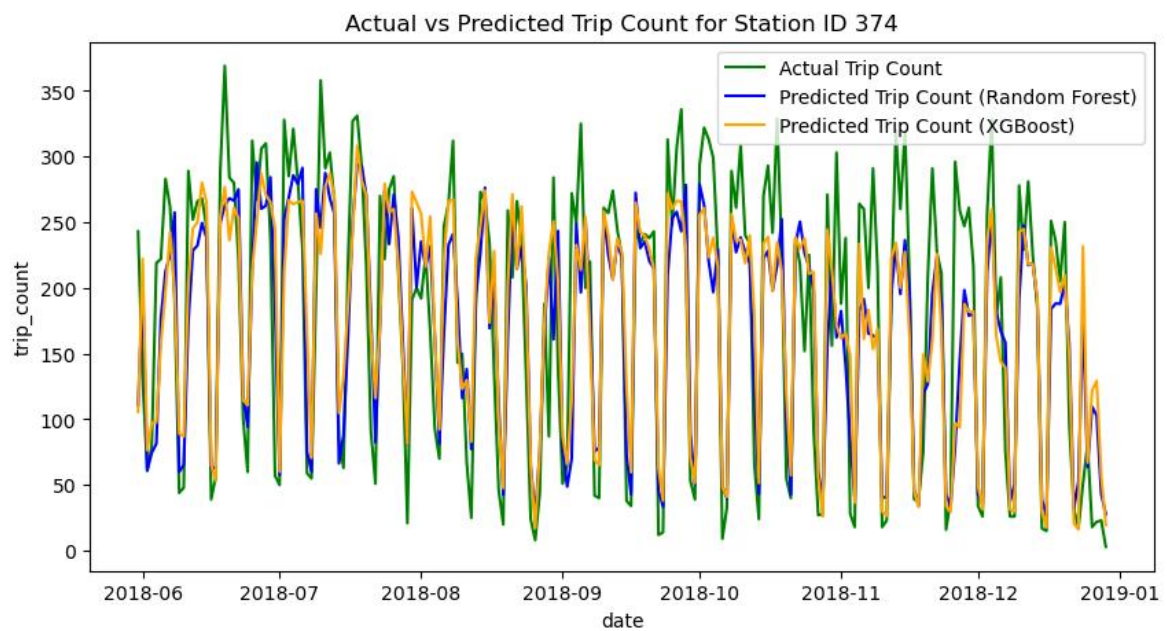
Rys. 26 Porównanie wyników dla stacji id 303: Albert Gate, Hyde Park

5.4.1 Stacja id 307: Black Lion Gate, Kensington Garden



Rys. 27 Porównanie wyników dla stacji id 307: Black Lion Gate, Kensington Garden

5.4.2 Stacja id 374: Waterloo Station 6, Waterloo Station



Rys. 28 Porównanie wyników dla stacji id 374: Waterloo Station 6, Waterloo Station

6. Podsumowanie i wnioski

Przyglądając się wykresom można zauważyć, że największe różnice między predykcjami, a wartościami rzeczywistymi liczby wypożyczeń pojawiają się w lokalnych minimach i maksymach funkcji wypożyczeń w czasie i są szczególnie wyraźne po 7-14 dniach od ostatniej znanej wartości liczby wypożyczeń na danej stacji. Może to świadczyć o tym, że modele typu Las Losowy czy XGBoost mogą wspierać prognozy krótkoterminowe, które mogą wspierać decyzje operacyjne takie jak relokacja rowerów czy informacje o dostępności rowerów na danej stacji. Wydajniejszym modelem jest XGBoost, który wraz z optymalizacją hiperparametrów zajmuje zaledwie 3% czasu, który jest potrzebny na optymalizację Lasu Losowego. W trakcie opracowywania niniejszej pracy, obliczenia dla XGBoost zajęły 56,1 sekundy, w porównaniu do 29 minut i 7,7 sekundy dla Lasu Losowego, jednocześnie osiągając lepsze wyniki.

6.1 Wnioski:

1. Modele drzewiaste typu Random Forest czy gradient boosting wykazują się dobrym dopasowaniem predykcji do danych rzeczywistych.
2. Wraz ze wzrostem odstępu czasu od ostatnich znanych wartości zmiennej celu można zauważyć pogarszające się dopasowanie wartości, w szczególności w lokalnych minimach i maksymach.
3. W związku z powyższym modele Las Losowy czy XGBoost mogą być skuteczne w krótkookresowych predykcjach, co w przypadku systemów wypożyczalni rowerów miejskich może być wykorzystywane do działań operacyjnych.
4. Model XGBoost wykazuje się znacznie większą wydajnością obliczeniową od modelu Lasu Losowego. Minimalnie lepsze wyniki wspierają tezę, że model XGBoost Regressor jest lepiej dopasowany do krótkoterminowej predykcji popytu na rowery miejskie w Londynie.

7. Źródła danych

1. **Dane o systemie wypożyczeń rowerów miejskich w Londynie**, publiczny zbiór danych London Bicycles, Google BigQuery, https://console.cloud.google.com/bigquery?ws=!1m4!1m3!3m2!1sbigquery-public-data!2slondon_bicycles, 2025.05.05
2. **Dane o wypadkach z udziałem rowerzystów w Londynie w latach 2005-2018**, publiczny zbiór danych London Cycling Safety, Kaggle.com <https://www.kaggle.com/datasets/dtuthill/london-cycling-safety>, 2025-05-05
3. **Dane pogodowe 2016.01.31 – 2018.12.31**, <https://www.visualcrossing.com/weather-query-builder/#>, 2025-05-05

8. Załączniki do pracy

1. Załącznik 1: *Prognozowanie_popytu_rowery_miejskie_Londyn.ipynb*
2. Załącznik 2: zbiór danych *london_bikes.csv*
3. Załącznik 3:

9. Spis literatury

Walker, P., *Jak rowery mogą uratować świat*, Wysoki Zamek, 2018

Bruce, P., Bruce A., Gedeck P., *Statystyka praktyczna w data science. 50 kluczowych zagadnień w językach R i Python*, Helion SA, 2021

Korczyński, A., *Modelowanie statystyczne dla biznesu. Teoria i zastosowania z wykorzystaniem SAS Viya, R i Python*, Oficyna Wydawnicza SGH, 2023

Vaughan, D., *Data science. Wyzwania i rozwiązania. Jak zostać ekspertem analizy danych*, Helion SA, 2025

Gallatin, K., Albon, Ch., *Uczenie maszynowe w Pythonie. Receptury. Od przygotowania danych do deep learningu*, Helion SA, 2024

<https://pandas.pydata.org/docs/>, 2025.05.05 – 2025.06.19

<https://scikit-learn.org/stable/index.html>, 2025.05.05 – 2025.06.19

<https://seaborn.pydata.org/index.html>, 2025.05.05-2025.06.19

10. Spis ilustracji

Rys. 1 Widok pięciu pierwszych wierszy wgranych danych	12
Rys. 2 Podstawowe statystyki danych numerycznych	14
Rys. 3 Liczebność występowania zmiennej 'trip count'	15
Rys. 4 Liczebność występowania zmiennej 'fatal_acc'	16
Rys. 5 Liczebność występowania zmiennej 'serious_acc'	16
Rys. 6 Liczebność występowania zmiennej 'slight_acc'	16
Rys. 7 Liczebność występowania zmiennej 'num_acc'	17
Rys. 8 Liczebność występowania zmiennej 'temp1'	17
Rys. 9 Liczebność występowania zmiennej 'temp2'	17
Rys. 10 Liczebność występowania zmiennej 'hum'	18
Rys. 11 Liczebność występowania zmiennej 'wind_speed'	18
Rys. 12 Liczebność występowania poszczególnych wartości zmiennej skokowej 'weekday'	19
Rys. 13 Liczebność występowania poszczególnych wartości zmiennej skokowej 'weather_code'	19
Rys. 14 Liczebność występowania poszczególnych wartości zmiennej skokowej 'season'	20
Rys. 15 Liczebność występowania poszczególnych wartości zmiennej skokowej 'is_holiday'	20
Rys. 16 Liczebność występowania poszczególnych wartości zmiennej skokowej 'is_weekend'	20
Rys. 17 Wykres liniowy liczby wypożyczeń w zależności od dnia tygodnia i pory roku ..	21
Rys. 18 Wykres liniowy liczby wypożyczeń w zależności od dnia tygodnia i roku	21
Rys. 19 Wykres liniowy wypożyczeń przez cały zakres czasowy	22
Rys. 20 Tabela korelacji dla zmiennych numerycznych	23
Rys. 21 Heatmapa korelacji dla zmiennych numerycznych	23
Rys. 22 Porównanie metryk obu modeli	33

Rys. 23 Istotność cech dla obu modeli	34
Rys. 24 Porównanie wyników dla stacji id 191: Hyde Park Corner, Hyde Park	35
Rys. 25 Porównanie wyników dla stacji id 154: Waterloo Station 3, Waterloo Station.....	36
Rys. 26 Porównanie wyników dla stacji id 303: Albert Gate, Hyde Park	36
Rys. 27 Porównanie wyników dla stacji id 307: Black Lion Gate, Kensington Garden	37
Rys. 28 Porównanie wyników dla stacji id 374: Waterloo Staton 6, Waterloo Station.....	37

11. Spis tabel

Tab. 1 Wyniki metryk ewaluacyjnych dla modelu RandomForestRegressor na zbiorze testowym.....	25
Tab. 2 Wyniki metryk ewaluacyjnych dla modelu RandomForestRegressor na zbiorze testowym.....	28
Tab. 3 Wyniki metryk ewaluacyjnych dla modelu RandomForestRegressor na zbiorze testowym dla najlepszych parametrów	29
Tab. 4 Wyniki metryk ewaluacyjnych dla modelu XGBoostRegressor na zbiorze testowym	30
Tab. 5 Wyniki metryk ewaluacyjnych dla modelu XGBoostRegressor na zbiorze testowym dla najlepszych parametrów	32
Tab. 6 Wyniki ewaluacji końcowych modeli RandomForestRegressor i XGBoostRegressor	32